Лабораторная работа №05

Наложение текстур с использованием GLSL-шейдеров.

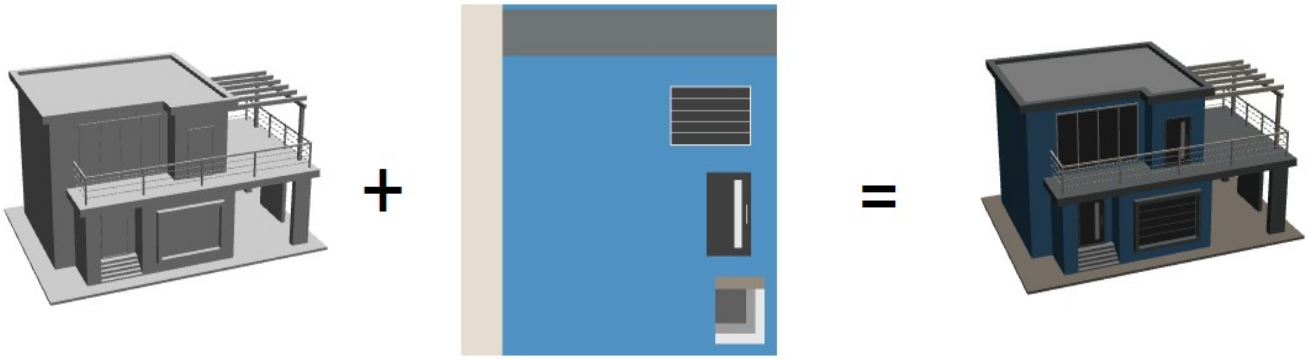
Целью лабораторной работы №5 является вывод графических объектов с наложенными на них текстурами. Для наложения текстур необходимо использовать специальный glsl-шейдер, реализующий данную функциональность, а также корректно настроить конвейер рендеринга. Для удобства работы с текстурами необходимо создать отдельный класс Texture, а для загрузки текстур из внешних файлов в оперативную память рекомендуется использовать библиотеку DevIL.

Некоторым изменениям в данной лабораторной работе подвергнуться также классы GraphicObject и ResourceManager для того, чтобы отразить добавление текстуры, которая, с одной стороны, является ресурсом программы и должна быть загружена через менеджер ресурсов, а, с другой стороны, является частью графического объекта.

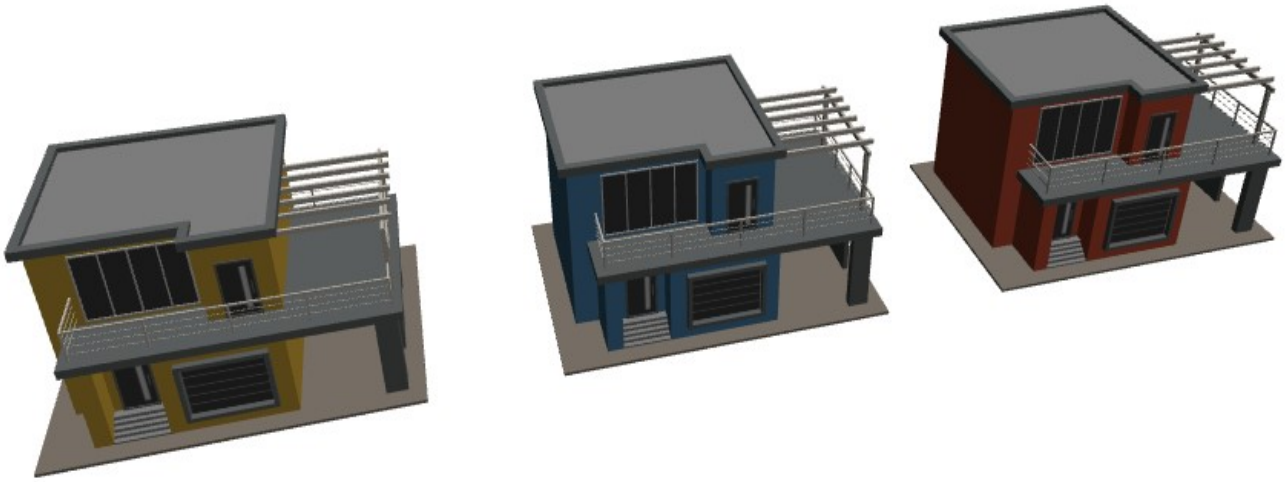
Еще одной целью лабораторной работы является организация класса RenderManager для централизованного управления выводом всех графических объектов на экран. Данный класс берет на себя обязанность по взаимодействие с видеокартой, скрывая от остальных модулей детали этого взаимодействия. В результате другие модули могут сосредоточиться на выполнении иных задач, таких как, например: считывание позиции графических объектов по сети или принятие решения о том, какие объекты требуется выводить на экран в соответствии с логикой работы программы или алгоритмами оптимизации.

Введение в использование текстур в OpenGL и GLSL-шейдерах.

Традиционно под текстурой понимается двухмерное изображение, накладываемой на объект на манер обоев, для придания каждому отдельному фрагменту объекта своего собственного цвета. При этом значение из текстуры может по-разному влиять на итоговый цвет объекта. Наиболее часто ранее вычисленный цвет комбинируется с цветом, полученным из текстуры, для того чтобы одновременно учесть и раскраску объекта (задаваемую текстурой) и условия освещения (вычисляемые, например, в соответствии с моделью освещения по Фонгу, что будет реализовано в следующей лабораторной работе):



Преимуществом использование текстур является не только возможность задать индивидуальный цвет фрагмента, но и возможность получить несколько объектов, имеющих разную раскраску, используя один и тот же меш, но разные текстуры:



Следует отметить, что для того, чтобы указать соответствие между отдельным полигоном меша и областью текстуры, которая будет накладываться именно на этот полигон, каждая вершина полигона должна обладать текстурными координатами. Таким образом, если на объект планируется накладывать текстуры, помимо прочих атрибутов необходимо также передавать текстурные координаты для каждой вершины.

Наложение текстур в программируемом конвейере рендеринга OpenGL осуществляется посредством использования трех основных сущностей (объектов OpenGL и GLSL-шейдеров), каждый из которых реализует свою часть функциональности. Некоторые из этих сущностей использовались и в фиксированном конвейере рендеринга, который был рассмотрен в предыдущем семестре, но в программируемом конвейере рендеринга претерпели некоторые изменения:

1. **Текстурный объект (texture)** – это внутренний объект OpenGL который хранит в себе исходное изображение текстуры (массив текселей), все уровни мипмапов (уменьшенные отфильтрованные копии исходной текстуры), а также некоторые дополнительные параметры текстуры, такие как, например: режим фильтрации текстуры (texture filtering mode) и режим адресации текселей (texture wrap mode).

   Введение текстурного объекта было необходимым для того, чтобы можно было хранить саму текстуру в памяти видеокарты, скрыв от разработчиков сложности или особенности хранения для различных видеокарт, предоставив унифицированный интерфейс взаимодействия с текстурой. Для идентификации текстуры, то есть для указания того, какую текстуру необходимо использовать в том или ином случае, используется её идентификатор, представленный беззнаковым целым.

   Графические программы в процессе своей работы используют множество текстур, накладываемых на разные объекты. Каждая из этих текстур загружается из внешней памяти (из файла графического формата) в текстурный объект. Таким образом, можно считать, что текстурный объект является представлением исходной текстуры (самого изображения, которое можно просмотреть в графическом редакторе) в памяти видеокарты.

2. **Текстурный блок (texture unit)**– это внутренний, предопределенный объект OpenGL, необходимый для того, чтобы указать видеокарте какой текстурный объект из множества ранее загруженных необходимо использовать для наложения на графический объект в момент его вывода на экран.

   В процессе своей работы приложение может загружать несколько сотен текстур, по одной на каждую модель, используемую в программе, однако для наложения на один конкретный объект используются только некоторые из них. Для этого видеокарта поддерживает заранее определенное количество текстурных блоков (не менее восьми) к каждому из которых можно привязать один из текстурных объектов, указав, таким образом, что для вывода последующего объекта будут использоваться именно эти текстуры.

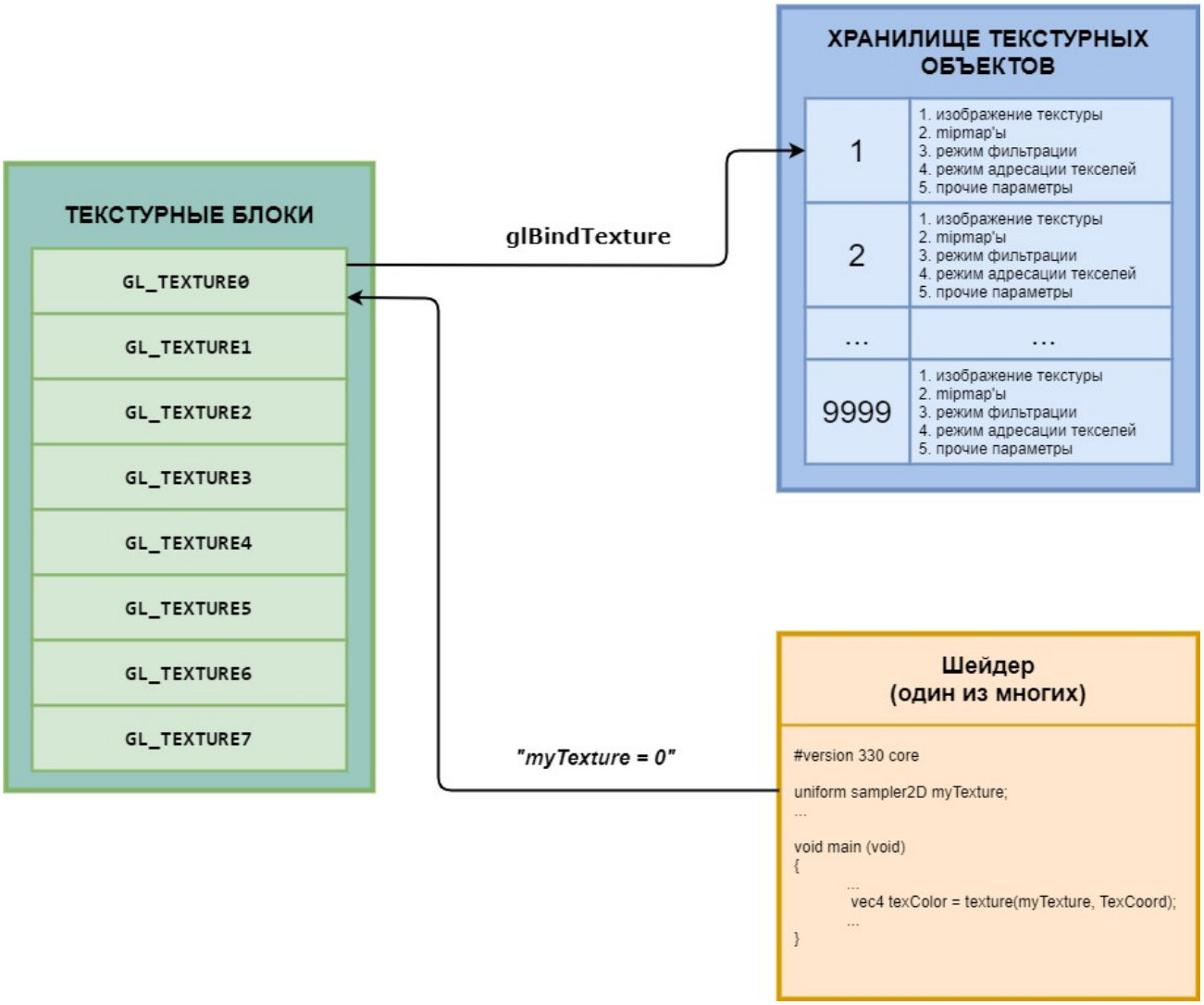
   В фиксированном конвейере рендеринга, рассмотренном в предыдущем семестре, текстурный блок имел также и другие параметры, важнейшим из которых являлся режим наложения текстуры. Режим наложения текстуры отвечал за то, как значение из текстуры будет комбинироваться с ранее вычисленным цветом фрагмента. В частности, были доступны такие режимы, как режим модуляции (GL_MODULATE) или режим замещения (GL_REPLACE).

   В программируемом конвейере рендеринга режим наложения текстуры признан устаревшим и не используется, а способ комбинирования определяется с помощью математических формул в самом шейдере. В этом заключается одно из преимуществ использования шейдеров.

3. **Сэмплер (sampler)** – это специальный встроенный тип данных GLSL для доступа к текстурам из шейдеров. Для обращения к текстурному объекту необходимо объявить в шейдере uniform-переменную типа sampler и присвоить ей номер одного из текстурных блоков. В результате, при использовании этой переменной, будет получено значение из того текстурного объекта, который был ранее привязан к текстурному блоку с указанным номером.

   В некотором роде sampler является аналогом дескриптора файла. Переменная этого типа не является непосредственно самой текстурой, но позволяет её идентифицировать, скрыв от разработчика несущественные для него детали реализации и обеспечив дополнительный контроль со стороны видеокарты. Для получения значения из текстуры необходимо использовать одну из встроенных функций GLSL, параметром которой является переменная типа sampler. Подробнее про получение значения из текстуры в GLSL-шейдере будет рассказано в одном из последующих разделов.

Взаимосвязь всех трех сущностей, участвующих в наложении текстур, может быть проиллюстрирована следующим рисунком:



Следует отметить, что в программе имеется множество текстур (текстурных объектов) каждый из которых идентифицируется беззнаковым целым. Для того, чтобы наложить текстуру на объект, её необходимо вначале привязать к одному из текстурных блоков, использую функцию glBindTexture. На изображении сверху текстурный объект с идентификатором [1] привязывается к нулевому текстурному блоку.

В свою очередь, шейдер для доступа к двухмерной текстуре использует переменную myTexture типа sampler2D. Эта переменная является uniform-переменной, которой из основной программы присвоено целочисленное значение ноль, что говорит о том, что переменная связанна с нулевым текстурным блоком.

Для получения значения из текстуры в GLSL-шейдере используется встроенная функция texture, первым параметром которой является переменная типа sampler. При этом происходит обращение не к какому-то конкретному текстурному объекту, а к текстурному блоку с номером, соответствующим значению этой переменной. Вторым параметром функции являются текстурные координаты, которые показывают из какого конкретно места текстуры следует извлекать данные.

В текстурном блоке определяется с каким текстурным объектом он связан в настоящий момент, и запрос адресуется соответствующему текстурному объекту. Здесь текстурный блок выступает своего рода точкой привязки, связывающей текстурный объект и переменную типа sampler2D. Данный подход используется достаточно часто, поэтому на него необходимо обратить особое внимание.

В завершении следует отметить, что значение выбирается из текстурного объекта в соответствии с режимом фильтрации и режимом адресации текселей. Данное значение является результатом работы функции texture, представляет собой вектор из четырех компонент (r, g, b, a) и может использоваться в шейдере абсолютно произвольным образом.

Суммируя все вышесказанное, можно привести схематичный план вывода объектов с наложенной на них текстурой. Более подробная версия некоторых этапов будет рассмотрена далее, в соответствующих разделах:

1. **Создание текстурного объекта.** Прежде всего необходимо создать текстурный объект, загрузить в него изображение и установить различные параметры, важнейшим из который, пожалуй, является режим фильтрации. Данный этап выполняется, как правило, один раз при инициализации программы или при первом обращении к текстуре.
2. **Выбор шейдера.** Перед выводом объекта, как и ранее, необходимо выбрать шейдер, который будет использоваться для его отображения. Для наложения двухмерной текстуры этот шейдер должен определить uniform-переменную типа sampler2D с помощью которой будет осуществляться доступ к текстуре. Как именно использовать значение, полученное из текстуры, решает сам шейдер.
3. **Привязка текстуры к текстурному блоку.** Перед выводом объекта необходимо привязать одну из текстур к какому-либо текстурному блоку, поскольку эта привязка определяет, какая именно текстура из множества ранее загруженных будет использоваться для отображения именно этого графического объекта.
4. **Установка значения uniform-переменных.** Далее требуется установить значение всех uniform-переменных, необходимых для работы шейдера. При использовании двухмерной текстуры необходимо, в том числе, установить значение uniform-переменной типа sampler2D, присвоив ей номер текстурного блока, с которым будет связанная эта переменная. Поскольку в данной лабораторной работе на все объекты накладывается только одна текстура, то логично использовать только нулевой текстурный блок, поэтому значение uniform-переменной типа sampler2D должно принимать значение ноль.
5. **Вывод меша.** Завершающим этапом вывода объекта является вывод его меша, то есть передача в OpenGL всех атрибутов каждой вершины. При наложении текстуры, помимо прочих атрибутов, требуется, как правило, передавать текстурные координаты для каждой вершины.

Работа с текстурным объектом.

Работу с текстурным объектом можно разбить на два основных этапа: создание текстурного объекта с его последующей инициализацией и привязка текстурного объекта к текстурному блоку с целью его использования при отрисовки модели. Стоит отметить, что работа с текстурным объектом уже была реализована в прошлом семестре и во многом осталось неизменной. Тем не менее, в современной версии OpenGL существуют определенные отличия, которые необходимо учесть. Речь, прежде всего, идет о способе генерирования mipmap уровней текстуры, без которых невозможно использовать трилинейную или анизотропную фильтрацию.

Создание текстурного объекта.

Для представления одной текстуры необходимо создать текстурный объект, загрузить в него массив текселей исходного изображения, сгенерировать все уровни mipmap'ов, а также установить различные параметры текстуры. Данный процесс осуществляется, как правило, один раз и выполняется в соответствии со следующей последовательностью действий:

1. Загрузка изображения из внешнего файла в оперативную память. Данная функциональность напрямую не относится к OpenGL и должна быть выполнена самостоятельно. В частности, рекомендуется использовать библиотеку DevIL. Не следует забывать о необходимости инициализировать эту библиотеку! Использование данной библиотеки было подробно рассмотрено в проекте прошлого семестра.
2. Выбор активного текстурного блока – выполняется с помощью функции glActiveTexture. Большая часть функций OpenGL, которые работают с текстурами, использует именно активный текстурный блок.
3. Создание нового текстурного объекта – выполняется с помощью функции glGenTextures. Текстурный объект содержит все параметры текстуры, включая её изображение (массив текселей). Каждый текстурный объект идентифицируется беззнаковым целым числом.
4. Привязка текстурного объекта к текстурному блоку – выполняется с помощью функции glBindTexture. Использование текстурного объекта (в том числе, загрузка в него изображения) возможно только если он привязан к текстурному блоку.
5. Установка параметров передачи пиксельных данных – выполняется с помощью функции glPixelStorei. Необходимо, как минимум, установить способ выравнивания пиксельных данных в оперативной памяти (параметр GL_UNPACK_ALIGNMENT = 1).
6. Загрузка изображения текстуры в текстурный объект – выполняется с помощью функции glTexImage2D. Следует отметить, что загрузка происходит в тот текстурный объект, который является привязанным к активному текстурному блоку в момент вызова данной функции.
7. Следом идет генерация всех уровней mipmap'ов. Mipmap уровни необходимы для некоторых режимов фильтрации текстуры. В случае если mipmap'ы отсутствуют (то есть не были сгенерированы), изображение будет абсолютно черным. В современной версии OpenGL (3.0 и выше) для генерации mipmap'ов используется специальная функция glGenerateMipmap!
8. Здесь же можно установить параметры текстуры с помощью команды glTexParameteri. У каждого текстурного объекта есть множество параметров, например: режим фильтрации текстур или режим адресации текселей.

Привязка текстурного объекта к текстурному блоку.

Привязка необходима для того, чтобы показать, что на объект, выводимый в дальнейшем, из всего множества ранее загруженных текстур необходимо накладывать именно эту текстуру. Привязка текстурного объекта к текстурному блоку, строго говоря, уже выполнялась ранее, на этапе загрузки текстуры, и осуществляется в соответствии со следующей последовательностью действий:

1. Выбор активного текстурного блока с помощью функции glActiveTexture.
2. Привязка текстурного объекта к текстурному блоку с помощью функции glBindTexture.
3. Установка, если необходимо, новых значений параметров текстурного объекта. Данный этап может быть необходим если, например, изменился режим фильтрации текстур. Изменение параметров текстурного объекта выполняется с помощью функции glTexParameteri.

Класс для работы с текстурой.

Для работы с текстурой необходимо реализовать отдельный класс, имеющий структуру, представленную ниже. При необходимости в класс можно добавлять дополнительные поля и методы, например, для смены режима фильтрации:

```
// КЛАСС ДЛЯ РАБОТЫ С ТЕКСТУРОЙ
class Texture
{
public:
	// загрузка текстуры из внешнего файла
	void load (std::string filename);
	// применение текстуры (привязка к текстурному блоку)
	void bind (GLenum texUnit = GL_TEXTURE0);

private:
	// индекс текстурного объекта
	GLuint texIndex;

};
```

Несколько замечаний по поводу использования текстур в проекте.

Необходимо сделать несколько замечаний, касающихся использования текстур в рамках конкретно данного проекта и о взаимосвязи текстур с остальными классами:

1. Текстура – это ресурс, поэтому работа с текстурой должна строится также, как и работа с мешем. В частности, за загрузку текстур должен отвечать менеджер ресурсов, который следит за тем, чтобы одна и та же текстура не загружалась в память более одного раза. После загрузки текстуры ResourceManager возвращает идентификатор текстуры, который используется остальными модулями для доступа к этой текстуре.
2. Логически текстура является частью графического объекта наравне с мешем. Также как и для меша, в графическом объекте хранится не сама текстура, а её индекс в менеджере ресурсов. Иными словами, необходимо еще раз модифицировать класс GraphicObject, добавив в него идентификатор текстуры и методы для его чтения и записи.
3. При выводе каждого графического объекта на экран необходимо получить его идентификатор текстуры, используя этот идентификатор обратиться к менеджеру ресурсов за самой текстурой и привязать текстуру к текстурному блоку для последующего наложения на объект.

Использование текстурного объекта для наложения текстур.

После реализации класса для работы с текстурными объектами необходимо еще раз рассмотреть последовательность этапов для вывода на экран графического объекта с наложенной на него двухмерной текстурой:

1. Прежде всего, необходимо выбрать ранее загруженный шейдер, который реализует алгоритм наложения текстуры на объект. В рамках данной лабораторной работы необходимо использовать шейдер, приведенный в демонстрационном примере. Алгоритм работы шейдера также будет рассмотрен ниже в соответствующем разделе.

2. Далее необходимо установить значение всех uniform-переменных, которые не имеют отношения к наложению текстур, но которые, например, используются для преобразования вершин. К таким переменным относятся: матрица проекции и матрица наблюдения модели.

3. Затем необходимо привязать текстурный объект к нужному текстурному блоку, за что отвечает метод bind класса Texture. Текстурный объект можно привязать к любому текстурному блоку, но, поскольку в данном проекте на каждый объект накладывается только одна текстура, логичнее привязать её именно к нулевому текстурному блоку, для чего используется константа GL_TEXTURE0.

4. Далее необходимо установить значение uniform-переменной типа Sampler2D, объявленной в шейдере. Данный шаг необходим для того, чтобы указать, из какого текстурного блока будут извлекаться данные при обращении в шейдере к соответствующей переменной.

   С точки зрения OpenGL переменная типа Sampler2D является целым числом и для её установки используется функция glUniform1i, то есть значение uniform-переменной должно быть равно номеру текстурного блока (в данном случае, числу ноль).

5. Завершающим шагом является вывод необходимого меша. Вывод меша подразумевает, в том числе, передачу в OpenGL текстурных координат наравне с прочими атрибутами вершин.

Для лучшего понимая предназначения всех классов и способа их использования, рассмотрим пример инициализации и оперирования графическими объектами для получения изображения на экране. Для начала укажем основные глобальные данные, которые используются в примере к лабораторной работе:

```
// ИСПОЛЬЗУЕМАЯ КАМЕРЫ
Camera camera;
// ИСПОЛЬЗУЕМЫЙ ШЕЙДЕР (ПОКА ТОЛЬКО ОДИН)
Shader shader;
// СПИСОК ГРАФИЧЕСКИХ ОБЪЕКТОВ ДЛЯ ВЫВОДА НА ЭКРАН
vector<GraphicObject> graphicObjects;
```

Перед началом использования (то есть перед выводом объектов на экран) графические объекты необходимо инициализировать. Пример инициализации объектов был показан в прошлой лабораторной работе, здесь же необходимо внести некоторые изменения. В частности, необходимо для каждого графического объекта загрузить текстуру с помощью менеджера ресурсов и сохранить полученный идентификатор текстуры в самом графическом объекте. Пример инициализации одного графического объекта может выглядеть следующим образом:

```
...
// добавление одного графического объекта
meshId = rm.loadMesh("MESHES\\buildings\\house_2.obj");
graphicObject.setMeshId(meshId);
textureId = rm.loadTexture("TEXTURES\\buildings\\house_2_orange.png");
graphicObject.setTextureId(textureId);
graphicObject.setColor(vec4(0.2, 0.2, 0.2, 1));
graphicObject.setPosition(vec3(0, 0, 0));
graphicObject.setAngle(0.0);
graphicObjects.push_back(graphicObject);
...
```

С учетом ранее реализованного класса для работы с текстурой (Texture), а также классов, в которые были внесены необходимые изменения (ResourceManager и GraphicObject) функция для вывода на экран графических объектов с наложенными на них текстурами принимает следующий вид:

```
// функция вызывается при перерисовке окна
// в том числе принудительно, по команде glutPostRedisplay
void display()
{
	// очистка буфера кадра
	glClearColor (1.0, 1.0, 1.0, 1.0);
	glClear (GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT );

	// включение теста глубины (на всякий случай)
	glEnable(GL_DEPTH_TEST);

	// активируем шейдер, используемый для вывода объекта
	shader.activate();

	// устанавливаем матрицу проекции
	mat4& projectionMatrix = camera.getProjectionMatrix();
	shader.setUniform("projectionMatrix", projectionMatrix);

	// получаем матрицу камеры
	mat4& viewMatrix = camera.getViewMatrix();

	// выводим все объекты
	for (auto& graphicObject : graphicObjects) {

		// устанавливаем матрицу наблюдения модели
		mat4 modelViewMatrix = viewMatrix * graphicObject.getModelMatrix();
		shader.setUniform("modelViewMatrix", modelViewMatrix);

		// устанавливаем цвет
		shader.setUniform("color", graphicObject.getColor());

		// устанавливаем текстуру (привязываем к текстурному блоку)
		int textureId = graphicObject.getTextureId();
		Texture *texture = ResourceManager::instance().getTexture(textureId);
		if (texture != nullptr) texture->bind(GL_TEXTURE0);

		// uniform-переменная texture_0 связанна с нулевым текстурным блоком
		shader.setUniform("texture_0", 0);

		// выводим меш
		int meshId = graphicObject.getMeshId();
		Mesh *mesh = ResourceManager::instance().getMesh(meshId);
		if (mesh != nullptr) mesh->drawOne();
	}

	// меняем передний и задний буферы цвета
	glutSwapBuffers();

	// вычисление FPS и его вывод в заголовок окна
	char windowTitle[128];
	int FPS = getFps();
	sprintf_s(windowTitle, 128, "Laba_05 [%i FPS]", FPS);
	glutSetWindowTitle(windowTitle);

}
```

Следует обратить внимание, что за счет введения таких классов как Shader, Texture и Mesh, процесс построения изображения на экране может быть описан высокоуровневыми понятиями без необходимости вдаваться в детали реализации. В этом и заключается преимущество объектно-ориентированного подхода к программированию и необходимость в тщательном проектировании архитектуры программы.

Реализация класса RenderManager.

В данной лабораторной работе необходимо реализовать класс RenderManager, который отвечает за вывод на экран переданных ему объектов. Данный класс призван облегчить работу вышестоящим модулям, предоставив простой и понятный интерфейс для вывода произвольного количества объектов. Например, во многих играх существуют невидимые персонажи, необходимость вывода которых на экран определяется тем, являются ли они союзниками или нет. Если они являются союзниками, то данная модель выводится на экран, в противном случае нет:

Таким образом, вышестоящий модуль, не вдаваясь в детали того, как будут выводиться графические модели на экран, просто формирует список объектов, который необходимо вывести, после чего передает этот список нижележащему модулю, в данном случае этим модулем будет RenderManager.

В свою очередь, класс RenderManager, получив список объектов, которые необходимо вывести, а также некоторые дополнительные параметры (в частности, камеру), применяет все известные ему алгоритмы для оптимального вывода объектов на экран. Данный подход позволяет более тонко настраивать систему, например, на ходу меняя способ фильтрации текстур, алгоритмы сглаживания или другие настройки графики. Более того, в случае необходимости это позволит относительно безболезненно перейти на другую графическую библиотеку, например Direct3D или более раннюю/позднюю версию OpenGL.

При данном способе организации программы происходит отделение логики принятия решения о том, что объект необходимо вывести на экран, от низкоуровневых алгоритмов вывода. Более того, вышестоящие модули могут реализовывать различные алгоритмы оптимизации (например, Frustum Culling) и таким образом вообще не передавать на отрисовку модели, которые не попадают в область видимости наблюдателя. Если же объект был передан в класс RenderManager, то тот выполняет все необходимые действия для его вывода на экран. К таким действиям относятся, например, выбор необходимого шейдера, передача в шейдер требуемых параметров и непосредственно вывод меша.

Ниже приведен прототип класса RenderManager. Следует обратить внимание, что класс организован на основе шаблона singleton, поскольку совершенно очевидно, что только один объект должен управлять видеокартой, то есть вызывать функции OpenGL:

```
// КЛАСС ДЛЯ ВЗАИМОДЕЙСТВИЯ С OPENGL
// ВЕСЬ ВЫВОД ОСУЩЕСТВЛЯЕТСЯ ЧЕРЕЗ ЕДИНСТВЕННЫЙ ЭКЗЕМПЛЯР ДАННОГО КЛАССА
class RenderManager
{
public:
	// Статик-метод для получения экземпляра менеджера ресурса.
	static RenderManager& instance()
	{
		static RenderManager renderManager;
		return renderManager;
	}

	// инициализация объекта RenderManager, выполняется после инициализации OpenGL:
	// загрузка шейдеров, установка неизменных параметров и прочая инициализация
	void init();

	// Начало вывода очередного кадра (подготовка, очистка вектора графических объектов)
	void start();

	// установка используемой камеры
	void setCamera(Camera *camera);
	// добавление в очередь рендеринга очередного объекта для вывода
	void addToRenderQueue(GraphicObject& graphicObject);

	// завершение вывода кадра (основная работа)
	void finish();

private:
	// конструктор по умолчанию (приватный)
	RenderManager() {};
	// конструктора копирования нет
	RenderManager(const RenderManager& root) = delete;
	// оператора присваивания нет
	RenderManager& operator=(const RenderManager&) = delete;

private:
	// используемые шейдеры
	std::vector<Shader>  shaders;

	// указатель на камеру
	Camera* camera;

	// список графических объектов, которые необходимо вывести в данном кадре
	std::vector<GraphicObject> graphicObjects;

};
```

Назначение всех полей и методов должно быть понятно по их описанию и приведенным комментариям. Следует отметить, что теперь шейдер является внутренним полем класса RenderManager, поскольку единственное предназначение класса Shader — это определять алгоритм вывода объектов.

При использовании класса RenderManager функция Display может выглядеть так, как показано ниже. Можно заметить, что в этом случае легко прослеживается логика вывода объектов, не вдаваясь в детали того, как именно они будут выводиться на экране. В результате можно модифицировать данную последовательность, например, передавая на рендеринг только те объекты, которые лежат достаточно близко к наблюдателю:

```
// функция вызывается при перерисовке окна
void display()
{
	// для удобства определяем ссылку на RenderManager
	RenderManager& renderManager = RenderManager::instance();

	// начинаем вывод нового кадра
	renderManager.start();

	// добавляем в очередь все объекты, которые необходимо вывести
	for (auto& graphicObject : graphicObjects) {
		renderManager.addToRenderQueue(graphicObject);
	}

	// завершаем построение кадра
	renderManager.finish();

	// меняем передний и задний буферы цвета
	glutSwapBuffers();

	// вычисление FPS и его вывод в заголовок окна
	char windowTitle[128];
	int FPS = getFps();
	sprintf_s(windowTitle, 128, "Laba_05 [%i FPS]", FPS);
	glutSetWindowTitle(windowTitle);

}
```

GLSL-шейдер для наложение текстур.

С появлением шейдеров появилась возможность использовать текстуры не только для вычисления цвета, но и абсолютно произвольным способом, то есть, с точки зрения GLSL-шейдера, текстура представляет собой массив значений, для доступа к элементам которого, используется не индекс, а текстурные координаты. Также необходимо отметить, что помимо двухмерных текстур OpenGL поддерживает и другие типы текстур, в частности: одномерные, двухмерные, трехмерные и кубические. Кроме того, элементом текстуры может быть не только цвет, но и другие данные, например, значения глубины (что используется в алгоритме shadow mapping для построения теней). Тем не менее, в данной работе текстура будет использоваться именно для расчета цвета объекта.

Для вывода меша с наложенной на него текстурой необходимо не только создать текстурный объект и привязать его к одному из текстурных блоков, но и использовать специальный шейдер, который реализует эту функциональность.

В частности, к шейдеру предъявляются следующие требования:

1. В шейдере должна быть объявлена глобальная uniform-переменная типа sampler, которая будет использоваться для доступа к текстуре. При этом, в зависимости от того, текстуру какого типа планируется использовать, необходимо выбрать конкретный тип переменной. Ниже приведены некоторые из возможных типов данных:

Тип текстуры	Тип sampler'a	Точка привязки (тип текстуры) в текстурном блоке
Одномерная	sampler1D	GL_TEXTURE_1D
Двухмерная	sampler2D	GL_TEXTURE_2D
Трехмерная	sampler3D	GL_TEXTURE_2D
Кубическая	samplerCube	GL_TEXTURE_CUBE_MAP
…	…	…

   Дополнительная информация доступна по следующей ссылке:
   https://www.khronos.org/opengl/wiki/Sampler_(GLSL)

2. Сэмплер позволяет обратиться к текстуре, однако для получения конкретного значения необходимы текстурные координаты, которые показывают какое именно значение из текстуры будет получено. Как правило, текстурные координаты передаются в качестве атрибутов вершин. Для доступа к различным типам текстур необходимо использовать текстурные координаты разной размерности:

Тип текстуры	Вектор текстурных координат	Тип данных в GLSL
Одномерная	(s)	float
Двухмерная	(s, t)	vec2
Трехмерная	(s, t, p)	vec3
Кубическая	(s, t, p)	vec3
…	…	…

   Следует отметить, что данный шаг не является обязательным, поскольку в некоторых случаях текстурные координаты могут быть вычислены в самом шейдере на основании, например, геометрических координатах вершины.

3. Следующим шагом является получение значения из текстуры. Для этого используется перегруженная функция texture первым параметром которой является sampler, а вторым - текстурные координаты. В зависимости от первого параметра, то есть от типа текстуры, текстурные координаты должны иметь соответствующий тип. Например, для получения значения из двухмерной текстуры используется функция, объявленная следующим образом:

```
vec4 texture(sampler2D sampler, vec2 p);
```

Результатом работы функции является значение, считанное из текстурного объекта, привязанного к текстурному блоку, на который указывает переменная типа sampler.

Следует отметить, что полученное значение не обязательно является значением какого-либо конкретного текселя, а вычисляется с учетом режимов фильтрации, то есть может являться усредненным значением нескольких соседних текселей или даже mipmap'ов.

Также необходимо отметить влияние режимов адресации текселей (wrapping mode) в случае, если какой-либо компонент текстурных координат (s, t) выходит за пределы интервала [0.0 … 1.0].

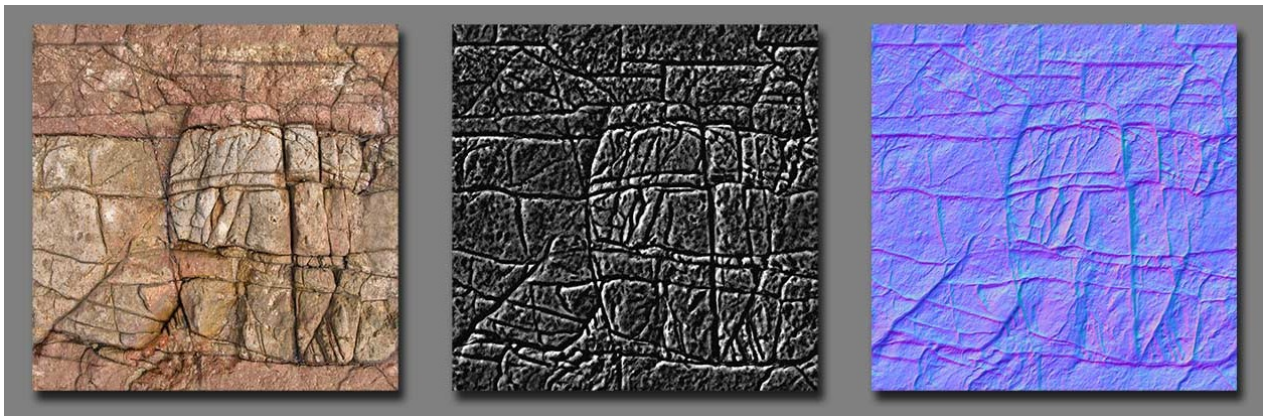
Кроме того, необходимо упомянуть, что в GLSL существуют и другие функции для работы с текстурой. Например, функция textureOffset позволяет получить значение из текстуры по заданным текстурным координатам, но с учетом смещения, заданного в целом количестве текселей, что может быть полезно для организации собственных режимов фильтрации или постобработки. Другой полезной функцией является функция textureSize, позволяющая получить размер текстуры в текселях. Дополнительную информацию по этим функциям можно получить на официальном сайте по следующим ссылкам:
https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/texture.xhtml
https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/textureOffset.xhtml
https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/textureSize.xhtml

4. В завершении необходимо каким-либо образом использовать значение, считанное из текстуры. Например, считанное значение можно умножить на цвет, вычисленный по формуле Фонга, что позволит одновременно учесть как индивидуальную раскраску фрагмента, заданную текстурой, так и его общую освещенность. Данный подход аналогичен режиму GL_MODULATE, используемому в фиксированном конвейере рендеринга.

   Поскольку значение, полученное из текстуры, является обычным вектором, оно может использоваться совершенно произвольным образом. В настоящее время фактическим стандартом при текстурировании моделей является использование трех текстур:

   - Диффузная текстура (diffuse map) – задает индивидуальный цвет фрагмента и в шейдере умножается на диффузную компоненту освещения.
   - Зеркальная текстура (specular map) – позволяет определить отдельные области, где возможно формирование блика. Зеркальная текстура, как правило, задается в оттенках серого и в шейдере умножается на зеркальную составляющую освещения.
   - Карта нормалей (bump map или normal map) – позволяет задать индивидуальную нормаль для каждого отдельного фрагмента. Эта техника получила названия bump mapping (или normal mapping) и служит для имитации микрорельефа поверхности.

   На рисунке снизу приведен пример указанных текстур:



Реализация вершинного шейдера.

Современные версии OpenGL позволяют обращаться к текстурам как из вершинного, так и из фрагментного шейдера. Более того, существуют алгоритмы, которые используют текстуры для задания смещения вершин в вершинном шейдер (например, алгоритм displacement mapping).

Тем не менее, чаще всего текстуры используется для задания цвета или других параметров фрагментов, поэтому доступ к ним осуществляется из фрагментного шейдера. В этом случае задачей вершинного шейдера становится передача текстурных координат, заданных как атрибуты вершин, из вершинного во фрагментный шейдер.

В результате, шейдер, используемый в примере к данной лабораторной работе, выглядит практически так, как показано ниже. Незначительные изменения будут приведены в конце раздела.

```
1  #version 330 core
2
3  uniform mat4 projectionMatrix;
4  uniform mat4 modelViewMatrix;
5  uniform vec4 color;
6
7  layout(location = 0) in vec3 vPosition;
8  layout(location = 1) in vec3 vNormal;
9  layout(location = 2) in vec2 vTexCoord;
10
11 out vec2 texCoord;
12
13 void main ()
14 {
15     texCoord = vTexCoord;
16     gl_Position = projectionMatrix * modelViewMatrix * vec4 (vPosition, 1);
17 }
```

Реализация фрагментного шейдера.

Задачей фрагментного шейдера является считывание значение из текстуры и использование этого значения для определения цвета фрагмента. Далее приводится полный текст фрагментного шейдера:

```
1  #version 330 core
2
3  uniform vec4 color;
4  uniform sampler2D texture_0;
5
6  in vec2 texCoord;
7
8  out vec4 outputColor;
9
10 void main (void)
11 {
12     vec4 texColor = texture(texture_0, texCoord);
13     outputColor = texColor;
14 }
```

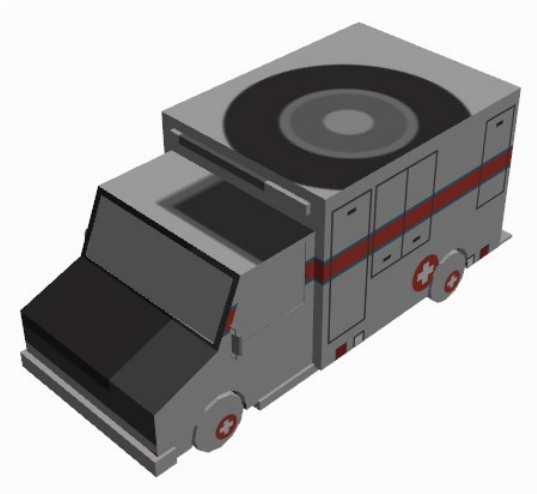
При рассмотрении шейдера необходимо обратить внимание на следующие моменты:

- Во фрагментном шейдере объявлена глобальная uniform-переменная texture_0 типа sampler2D, которая используется для доступа к текстурному объекту. Значение переменной устанавливается из основной программы и равно номеру текстурного блока (в данном случае - нулю).

- Также во фрагментном шейдере объявлена переменная texCoord типа vec2, которая содержит текстурные координаты и принимает значение из вершинного шейдера. Текстурные координаты для фрагмента получаются путем линейного интерполирования текстурных координат вершин, определяющих полигон, к которому относится фрагмент.
- Для считывания значения из текстуры используется функция texture, параметрами которой являются sampler и текстурные координат;
- Полученное значение используется для вычисления цвета фрагмента. В данной лабораторной работе итоговый цвет фрагмента равен значению, считанному из текстуры, что эквивалентно режиму наложения GL_REPLACE стандартного конвейера рендеринга. В следующей лабораторной работе цвет из текстуры будет комбинироваться с цветом, вычисленным с использованием модели освещения Фонга.

Несколько замечаний.

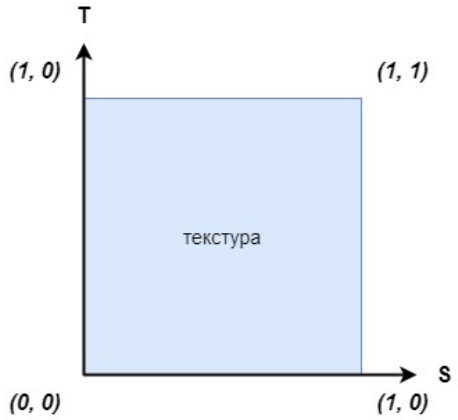
Результат применения рассмотренного шейдера, будет выглядеть не совсем нормально. Например, при наложении текстуры на модель машины скорой помощи, изображение будет выглядеть так, как показано в левой части рисунка, приведенного ниже:



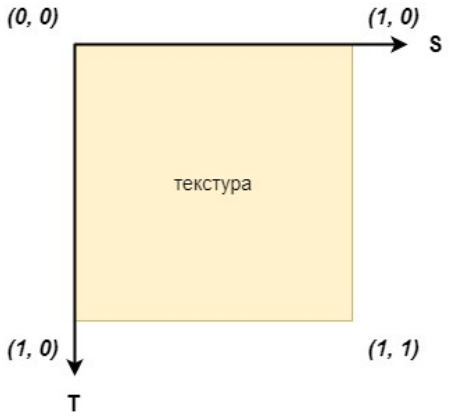
Некорректное наложение текстуры

Корректное наложение текстуры

Причина неправильного наложения текстуры заключается в том, что OpenGL считает, что текстурные координаты отсчитываются начиная с левого нижнего угла текстуры, в то время как программа, в которой моделировались модели, полагает, что координаты начинаются с левого верхнего угла текстуры:



Текстурные координаты в OpenGL



Текстурные координаты в графическом редакторе

Решить данную проблему достаточно просто, необходимо всего лишь преобразовать текстурную координату t по формуле t = 1.0 – t, при этом данная задача может быть решена тремя способами:

- Наиболее правильным вариантом является модификация obj-файлов на этапе разработки, для чего достаточно один раз пройтись по всем файлам, модифицировать текстурные координаты и пересохранить файл. Сделать это можно, например, написав небольшую утилиту. В результате миллионы пользователей, запуская программу сотни раз, не будут тратить машинное время на выполнение этой операции, а программа будет загружаться и выводить объекты на экран максимально быстро.
- Другим вариантом является автоматическая модификация текстурных координат при загрузке меша. В этом случае каждый пользователь выполняет эту операцию, однако операция выполняется только один раз при первом (после перезапуска программы) обращении к мешу. Таким образом, время загрузки увеличивается, но скорость вывода моделей на экран не страдает.
- Самым плохим способом является модификация текстурных координат каждый раз, при выводе графического объекта на экран. Делать это можно, например, в вершинном шейдере. В этом случае преобразования выполняются у каждого пользователя при построении каждого кадра, что, естественно, является крайне неэффективным решением. Хотя данный метод и не является оптимальным, в лабораторной работе будет использоваться именно он, для демонстрации того, что текстурные координаты могут быть преобразованы в шейдере произвольным образом, что является одним из преимуществ использования шейдеров:

Исправленная версия вершинного шейдера приведена ниже:

```
1  #version 330 core
2
3  uniform mat4 projectionMatrix;
4  uniform mat4 modelViewMatrix;
5  uniform vec4 color;
6
7  layout(location = 0) in vec3 vPosition;
8  layout(location = 1) in vec3 vNormal;
9  layout(location = 2) in vec2 vTexCoord;
10
11 out vec2 texCoord;
12
13 void main ()
14 {
15     texCoord = vec2(vTexCoord.s, 1.0 - vTexCoord.t);
16     gl_Position = projectionMatrix * modelViewMatrix * vec4 (vPosition, 1);
17 }
```

Задание к лабораторной работе.

Лабораторная работе №5 строится на основе предыдущей работы. К программе предъявляются следующие требования:

1. Необходимо реализовать класс Texture для работы с текстурой. Класс должен иметь структуру, приведенную в соответствующем разделе.
2. Необходимо внести изменение в класс ResourceManager так, чтобы работа с текстурой осуществлялась через ResourceManager. В частности, каждая текстура должна храниться в памяти в единственном экземпляре также, как это было сделано для меша.
3. Требуется модифицировать класс GraphicObject, добавив в него идентификатор используемой текстуры.
4. Необходимо реализовать класс RenderManager для централизованного вывода графических объектов на экран. Класс использует шаблон singleton и должен иметь структуру, приведенную ранее в соответствующем разделе.
5. В завершении требуется реализовать вывод на экран нескольких объектов с наложенными на них текстурами. Для демонстрации требуется вывести на консоль отладочную информацию, показывающую, что класс ResourceManager справляется со своей задачей, в частности, следит за тем, чтобы повторно не загружать уже загруженные меши и текстуры.
6. Ресурсы для демонстрации программы (меши и шейдеры) необходимо взять из демонстрационного примера.

Отчет к лабораторной работе должен быть сформирован на основе шаблона, представленного на следующей странице. При оформлении отчета необходимо внести свою фамилию, имя, отчество, номер группы, а также заполнить таблицу. При заполнении таблицы необходимо дать краткое описание реализованным модулям, указать количество строк каждого модуля, а также подсчитать общее количество строк во всем проекте.

Архитектура графических систем (6 семестр)

Выполнил: Иванов Иван Иванович

Группа: 00-00

Проверил: Галибин Сергей Владимирович

Задание:

В лабораторной работе №5 необходимо организовать вывод на экран нескольких графических объектов, на каждый из которых накладывается своя текстура. Для эффективного управления текстурами необходимо внести изменения в менеджер ресурсов, а для централизованного вывода графических объектов необходимо реализовать класс RenderManager.

Защита лабораторной работы:

Для защиты лабораторной работы необходимо знать:

1. **[КГ]** Что такое текстура и для чего она необходима в компьютерной графике.
2. **[КГ]** Что такое режим фильтрации текстуры, какие режимы фильтрации бывают.
3. **[КГ]** Для чего необходимы текстурные координаты.
4. **[КГ]** Какую роль в OpenGL выполняет текстурный объект.
5. **[КГ]** Какую роль в OpenGL выполняет текстурный блок.
6. **[КГ]** Какую роль в OpenGL/GLSL выполняет сэмплер (sampler2D).
7. **[КГ]** Взаимосвязь между текстурным объектом, текстурным блоком и сэмплером.
8. **[КГ]** Как происходит считывание значение из текстуры в шейдере GLSL.
9. **[CPP]** Для чего нужен и как реализуется шаблон singleton.
10. **[CPP]** Для чего необходим класс RenderManager.

Состав проекта:

Название модуля и его назначение	**Количество строк**	
	***.h**	***.cpp**
main - основная программа		
Shader -		
Camera -		
GraphicObject -		
Mesh -		
ResourceManager -		
Texture -		
RenderManager -		